

TERMINAL TALE

C++ 콘솔 텍스트 RPG

최종 보고서

Team 산 넘어 산

역할	이름	담당
팀장 / Core	이형원	Application·GameLoop·StateMachine·EventBus·Context·렌더링·화면 전환
저장 / 로드	노민지	SaveSlot·LoadSlot·QuickSlot·JSON 직렬화
UX / UI / 번역	홍성우	UIButton~UIScreenFader 8종 구현·5개 언어 번역
콘텐츠 / 데이터	김세민	스토리 JSON·아이템·업적·ConditionChecker·EffectInterpreter

1. 작품 개요 및 주제 선정 배경

1.1 프로젝트 소개

Terminal Tale은 C++로 구현된 콘솔(터미널) 기반 텍스트 RPG이다. 그래픽 엔진 없이 순수 콘솔 API만으로 192×54 크기의 화면을 구성하며, 플레이어는 국가 기관의 신입 직원이 되어 매일 사건 파일을 열람·처리한다. 플레이어의 선택은 도시 질서, 시민 신뢰, 오염도 등 세계 수치를 실시간으로 변화시키며, 누적된 성향(empathy·coldness·justice 등)이 스토리 분기와 선택지 잠금을 결정한다. 본 프로젝트의 핵심 목표는 두 가지다. 첫째, 그래픽 없이 서사 중심의 몰입감 있는 게임을 구현하는 것. 둘째, 대학 과제로서 C++ 객체지향 프로그래밍의 핵심 개념—상속, 다형성, 캡슐화, 합성—을 실제 게임 아키텍처에 적용하는 것이다.

1.2 세계관 및 장르 선택 이유

디스토피아 관료주의 세계관은 텍스트 RPG 장르와 구조적으로 잘 맞는다. 화려한 그래픽 없이도 도덕적 딜레마와 선택의 무게를 전달할 수 있으며, 파일 처리라는 반복적 행위 자체가 권력에 대한 복종·저항이라는 주제를 자연스럽게 담는다.

1.3 벤치마킹 자료

다음 네 작품을 참고하여 게임 디자인의 핵심 요소를 설계했다.

작품	연도	벤치마킹 포인트
Papers Please	2013	관료 시스템 세계관, 도덕적 딜레마 선택지, 피로·감시 스탯 (핵심 참고작)
Disco Elysium	2019	성향(tendency) 누적 시스템, 복수 수치 기반 선택지 잠금
Orwell	2016	파일 처리 UX, 감시 vs 시민 신뢰 긴장감
Roadwarden	2022	텍스트 중심 RPG UI/UX 레이아웃 구성 방식

Papers Please에서는 매일 케이스 파일을 처리하는 반복 구조와 피로도 시스템을, Disco Elysium에서는 선택이 누적되는 성향 시스템(감소 없이 증가만 하는 일방향 스탯)을 참고했다. Orwell은 파일 열람 UX와 감시 기관 긴장감의 표현 방식을, Roadwarden은 텍스트 RPG에서 레이아웃을 구성하는 방법을 참고했다.

1.4 과제 구현 목표

- **State** 패턴을 활용한 스택 기반 게임 상태 관리
- **Observer** 패턴의 타입 기반 **EventBus**로 모듈 간 결합도 최소화
- 추상 클래스(**UIElement**, **State**)와 다형성을 통한 확장 가능한 **UI/State** 계층 구성
- **JSON** 데이터 주도(**data-driven**) 방식으로 코드 수정 없이 스토리 콘텐츠 확장
- 전역 싱글톤 대신 **Context** 기반 의존성 주입으로 결합도 감소

2. 개발 환경

2.1 하드웨어·소프트웨어 구성

항목	내용
언어	C++ (ISO C++20 / /std:c++20)
IDE	Visual Studio 2022 / 2026
플랫폼 도구	MSVC v143 (Microsoft C++ Build Tools)
빌드 타겟	x64 Release / Debug (.exe)
실행 환경	Windows 10 이상 (x64)
버전 관리	Git & GitHub (PR 기반 협업)

2.2 외부 라이브러리

두 라이브러리 모두 헤더 온리(header-only) 형태로 `external/` 디렉터리에 포함되어 별도의 빌드 설정 없이 프로젝트에 통합되었다.

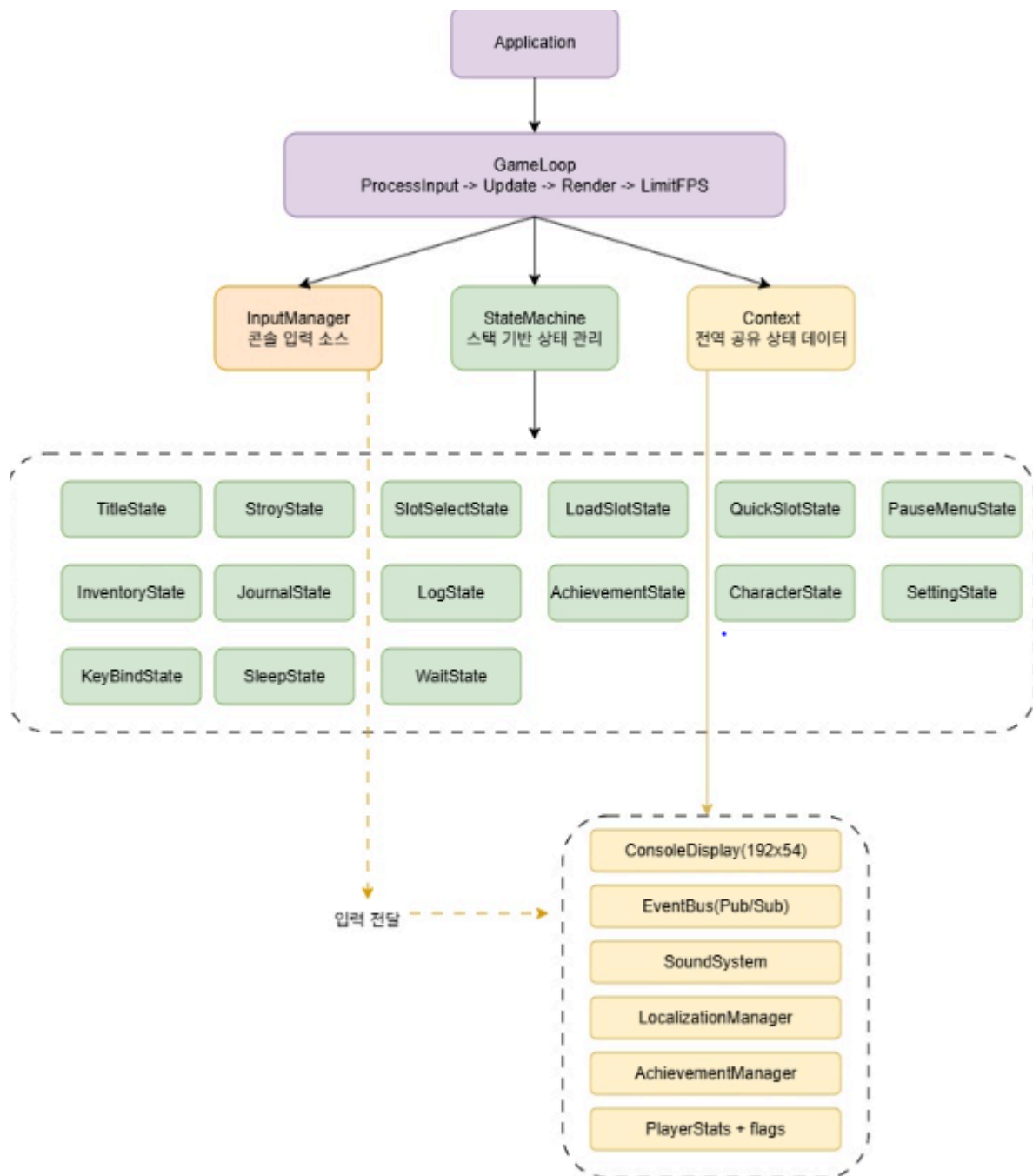
라이브러리	형태	라이선스	도입 이유
nlohmann/json	header-only	MIT	스토리·설정·세이브 파일을 JSON으로 관리. 직렬화·역직렬화가 직관적이고 유지보수가 쉬움
miniaudio	header-only	MIT / Public Domain	BGM 루프·SFX 단발 재생, 채널별 볼륨 독립 제어. 설치 없이 단일 헤더로 오디오 기능 구현

nlohmann/json을 선택한 결정적 이유는 C++ 표준 컨테이너와의 자연스러운 연동이다. `std::vector`, `std::string`, `std::unordered_map`과 직접 변환이 가능해 `StoryNode`, `PlayerStats`, `AchievementManager` 등 게임 데이터 구조의 직렬화 코드가 간결해진다. miniaudio는 단일 헤더로 크로스플랫폼 오디오를 지원하여 의존성 없이 WAV 재생이 가능했다.

3. 시스템 아키텍처 및 객체지향 설계

3.1 전체 구조

Terminal Tale의 구조는 `Application` → `GameLoop` → `StateMachine`의 수직 계층과, 모든 서브시스템을 공유하는 수평 컨테이너인 `Context`로 이루어진다. 아래 그림 1은 이 관계를 한눈에 보여준다.



▲ 그림 1. 전체 시스템 아키텍처

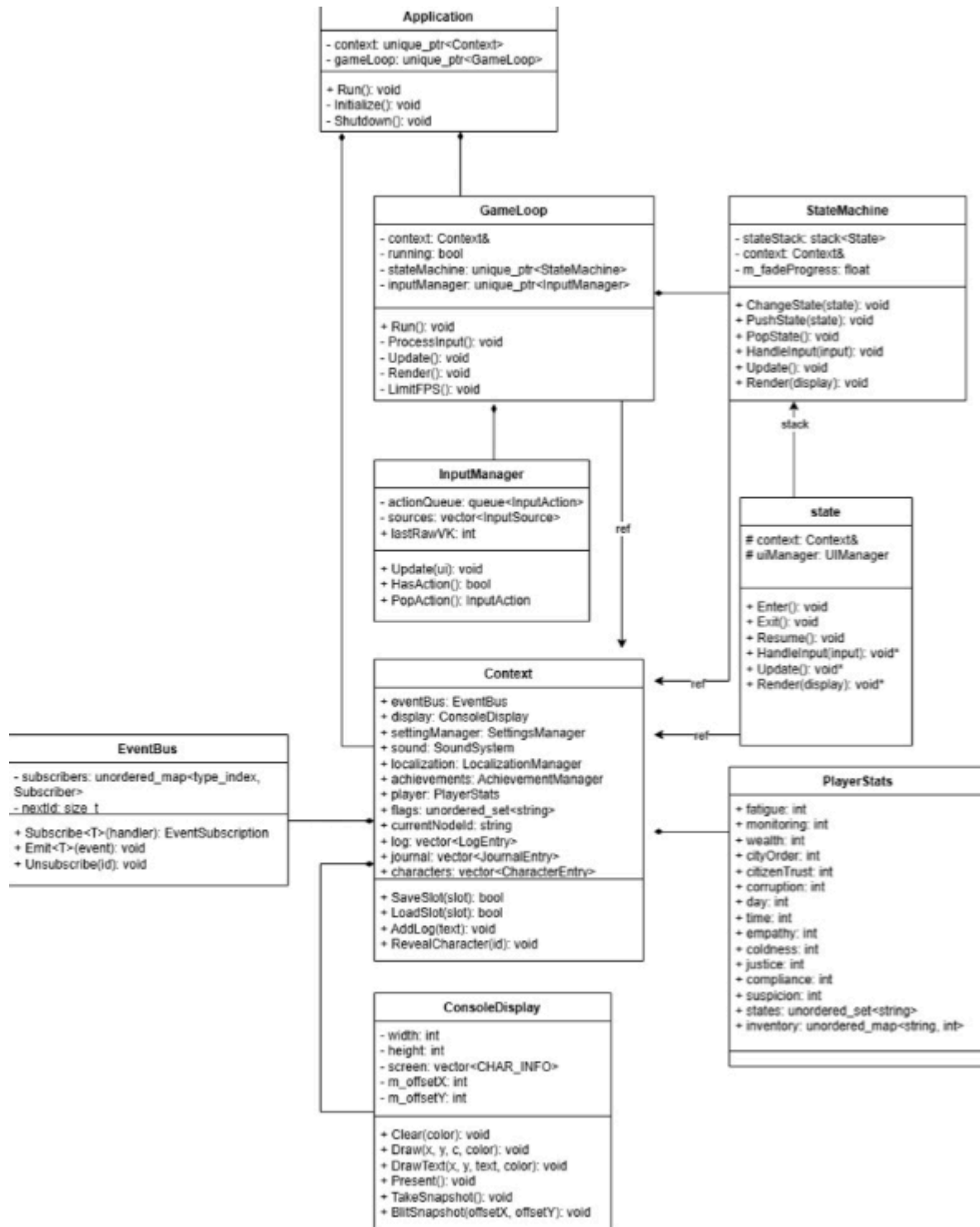
그림 1에서 **Application**은 **Context**와 **GameLoop**를 `unique_ptr`로 소유하여 생명주기를 관리한다. **GameLoop**는 매 프레임 **ProcessInput** → **Update** → **Render** → **LimitFPS** 순서로 동작하며, **StateMachine**에 입력·갱신·렌더 명령을 위임한다. **Context**는 모든 서브시스템(**ConsoleDisplay**, **EventBus**, **SoundSystem**, **LocalizationManager**, **AchievementManager**, **PlayerStats**)을 직접 포함(**composition**)하여 어느 **State**에서도 **Context** 참조 하나로 접근할 수 있도록 설계했다.

모듈	책임
Application	진입점. Context·GameLoop 초기화 및 Run() 루프 진입
GameLoop	프레임 루프 관리. ProcessInput / Update / Render / LimitFPS 순환
StateMachine	스택 기반 State 관리. ChangeState·PushState·PopState·화면 전환 페이드인
Context	전역 공유 컨테이너. 모든 서브시스템과 게임 데이터를 보유
InputManager	콘솔 키보드·마우스 입력 수집 및 actionQueue로 라우팅
ConsoleDisplay	192×54 CHAR_INFO 더블 버퍼. Present() 한 번으로 깜빡임 없이 출력
EventBus	타입 기반 Pub/Sub. 발행자·구독자 직접 의존 없이 이벤트 통신
SoundSystem	miniaudio 래퍼. BGM 루프·SFX 단발, 채널별 볼륨 독립 제어
LocalizationManager	5개 언어 JSON 로드. L("key") 매크로로 런타임 문자열 교체
AchievementManager	업적 해제·저장. 세이브 슬롯과 독립 유지
UIManager	UIElement 컨테이너. Z-order 정렬·마우스 라우팅·렌더 디스패치

3.2 클래스 다이어그램 해설

3.2.1 코어 아키텍처 (그림 2)

그림 2의 코어 아키텍처 클래스 다이어그램은 소유권과 참조 관계를 중심으로 설계 의도를 보여준다.

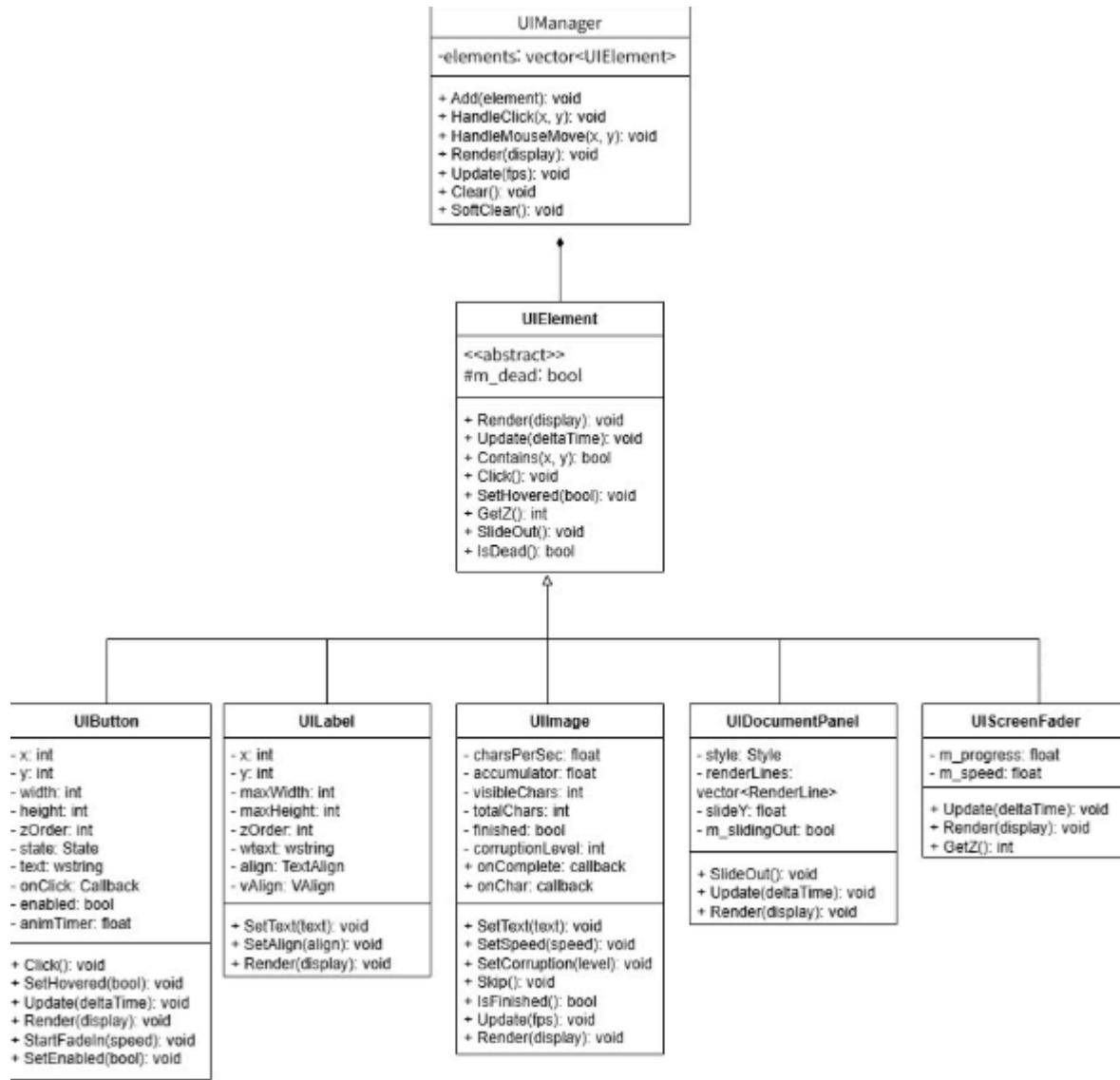


▲ 그림 2. 코어 아키텍처 클래스 다이어그램

Application은 Context와 GameLoop를 unique_ptr로 소유한다. Application이 전체 생명주기를 담당하고, GameLoop는 실행 흐름만을, Context는 데이터만을 관리한다. 각 State는 Context에 대한 참조(reference)만 보유하고 소유권을 갖지 않으므로, State가 교체되어도 공유 자원은 유지된다. EventBus는 Context 내부에 직접 포함(composition)되어 있어 별도 초기화 없이 사용 가능하다.

3.2.2 UI 계층 (그림 3)

그림 3은 UIElement 추상 클래스를 중심으로 한 다형성 구조를 보여준다.

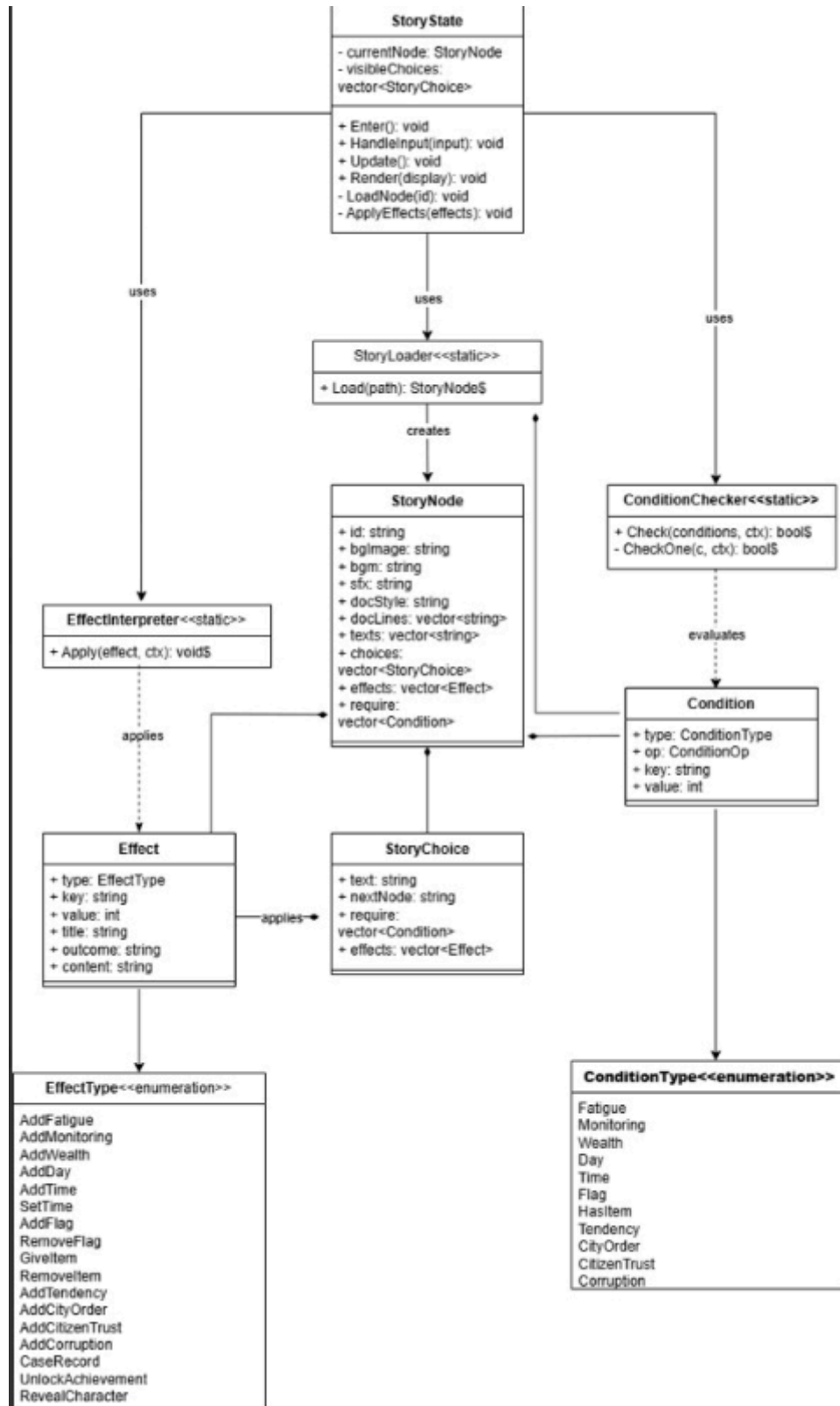


▲ 그림 3. UI 계층 클래스 다이어그램

UIElement는 Render(), Update(), Contains(), Click(), SetHovered(), GetZ(), IsDead() 등 6종의 순수 가상 메서드를 정의한다. UIManager는 UIElement 포인터의 벡터를 보유하며 Z값 오름차순으로 렌더링 순서를 결정한다. 새로운 UI 컴포넌트를 추가할 때 UIElement만 상속하면 UIManager는 수정 없이 그것을 처리할 수 있다.

3.2.3 게임 도메인 (그림 4)

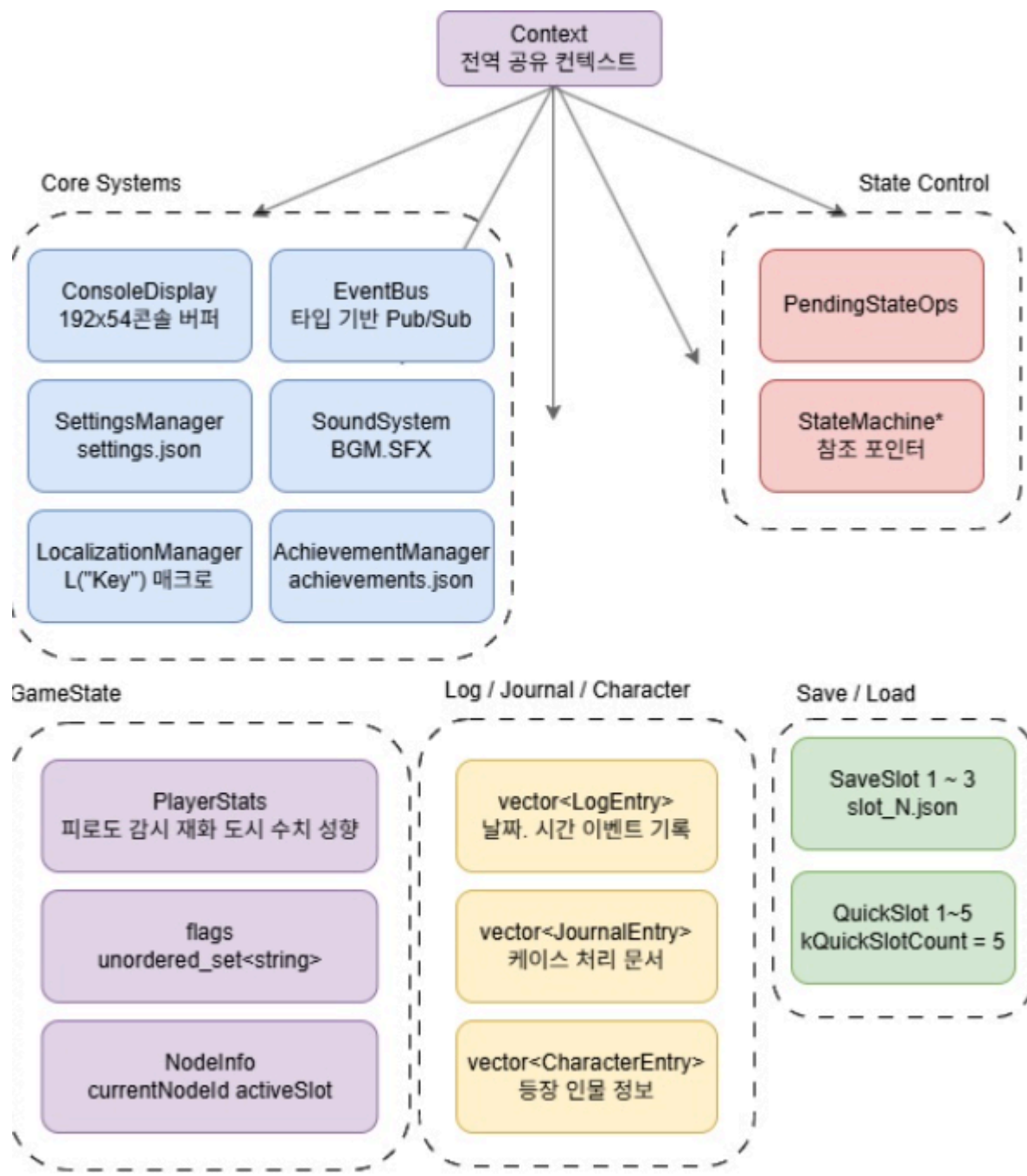
그림 4는 스토리 시스템의 협력 구조를 보여준다.



▲ 그림 4. 게임 도메인 클래스 다이어그램

StoryState는 StoryLoader, ConditionChecker, EffectInterpreter 세 정적 클래스와 협력한다. StoryLoader는 JSON 파일 하나를 StoryNode로 변환하는 역할만 담당하고, ConditionChecker는 조건 배열의 AND 평가만을, EffectInterpreter는 이펙트 적용만을 담당한다. 역할을 단일 책임으로 분리함으로써 각 클래스를 독립적으로 테스트·수정할 수 있다.

3.3 Context — 전역 공유 컨텍스트



▲ 그림 5. Context 내부 구성

그림 5는 **Context**가 담당하는 데이터와 시스템의 전체 범위를 보여준다. **Context**를 도입한 이유는 전역 싱글톤의 대안이다. 싱글톤은 테스트가 어렵고 의존성이 암묵적으로 형성된다. **Context**는 명시적 참조로 전달되므로 어떤 클래스가 무엇에 의존하는지가 코드에서 직접 드러난다.

Context에서 주목할 설계 포인트가 하나 있다. **pendingStateOps** 벡터다.

```
std::vector<PendingStateOp> pendingStateOps;
```

입력 처리 도중 즉시 **StateMachine**을 변경하면, 아직 살아 있는 **UIManager**가 파괴된 **State**에 접근하는 **use-after-free**가 발생한다. 이를 방지하기 위해 **State** 전환 요청을 큐에 쌓고, 프레임 끝(Update 이후)에 일괄 처리한다. 메모리 안전성과 실행 순서를 코드 구조로 보장한 사례다.

3.4 StateMachine — 스택 기반 상태 관리

StateMachine은 `std::stack<unique_ptr<State>>`를 사용하여 게임의 현재 맥락을 관리한다. 세 가지 전환 방식이 각각 다른 역할을 한다.

메서드	동작	사용 예
ChangeState(state)	스택 전체를 비우고 새 State 로 완전 전환	타이틀 → 인게임 전환
PushState(state)	현재 State 위에 새 State 를 올림 — 이전 State 유지	인게임 중 인벤토리 열기
PopState()	현재 State 를 제거하고 이전 State 복귀	인벤토리 닫고 StoryState 재개

구현된 **State** 목록은 다음과 같다.

State	역할
TitleState	타이틀 화면
StoryState	스토리 진행
SlotSelectState	저장 슬롯 선택
LoadSlotState	게임 불러오기

QuickSlotState	퀵 저장/로드
PauseMenuState	일시정지 메뉴
InventoryState	인벤토리
JournalState	처리 문서 열람
LogState	활동 로그 열람
AchievementState	업적 화면
CharacterState	인물 정보
SettingState	설정 화면
KeyBindState	키 바인딩
SleepState	수면/시간 경과
WaitState	대기 행동

PushState/PopState 직후 StartFade()가 자동으로 호출되어 m_fadeProgress를 0으로 리셋하고 약 0.25초($k\text{FadeSpeed} = 4.0f$) 동안 페이드인 애니메이션을 재생한다. RenderWithSlide()는 이전 프레임의 스냅샷과 현재 프레임을 합성하여 슬라이드 효과를 구현한다. 설정에서 screenTransition이 비활성화되어 있으면 페이드를 즉시 완료 처리하여 성능에 영향을 주지 않는다.

3.5 EventBus — 타입 기반 Pub/Sub

EventBus는 `std::unordered_map<std::type_index, vector<Subscriber>>`로 구독자를 관리한다. 실제 헤더 파일에서 확인되는 구현은 다음과 같다.

```
// 구독
auto sub = context.eventBus.Subscribe<PlaySoundEvent>(
    [this](const PlaySoundEvent& e) { Play(e.path); });

// 발행
context.eventBus.Emit(PlaySoundEvent{"Assets/audio/ui_click.wav"});
```

Subscribe<T>()는 EventType에 대해 typeid(EventType)을 키로 핸들러를 등록하고

EventSubscription(RAIL 래퍼)을 반환한다. EventSubscription이 소멸될 때 Unsubscribe()를 자동 호출하므로 메모리 누수 없이 구독이 해제된다. Emit<T>()는 동일 타입의 모든 구독자를 순회하여 호출한다. 발행자는 구독자가 누구인지 알 필요가 없고, 구독자는 발행자에 직접 접근하지 않는다. 이 구조로 SoundSystem, UIManager, StoryState 등 다양한 모듈이 이벤트 기반으로 통신하면서도 서로 의존하지 않는다.

3.6 UI 계층 — 다형성 기반 컴포넌트 시스템

UIElement 추상 클래스에서 파생된 6종의 컴포넌트가 각자의 역할을 담당한다.

클래스	핵심 특징
UIButton	호버 시 0.12초 확장→복귀 펄스 애니메이션·페이드인·Callback 클릭 처리
UILabel	가로(L/C/R) + 세로(T/M/B) 정렬·자동 줄바꿈
UIImage	아스키 아트 파일 로드 및 Wide 문자열 출력
UITypewriter	FPS 독립 글자별 출력(1~5단계, 6~60글자/초)·오염도 글자 깨짐 효과·onComplete/onChar 콜백
UIDocumentPanel	슬라이드 인/아웃 90행/초·Record / Order 스타일 2종
UIScreenFader	화면 전체 페이드인·Z=9999 최상위 렌더링

UIManager.Render()는 elements 벡터를 Z값 오름차순으로 정렬한 뒤 각 요소의 Render()를 가상 함수 디스패치로 호출한다. UIScreenFader는 Z=9999로 항상 최상위에 렌더링되어 화면 전환 시 다른 모든 요소를 덮는다. UI 요소가 SlideOut() 또는 IsDead()로 퇴장 상태가 되면 SoftClear() 호출 시 자동으로 제거된다.

3.7 데이터 주도 스토리 구조

스토리는 Data/story/ 디렉터리의 JSON 파일로 정의된다. 실제 저장소에는 case_1042부터 case_1069까지 약 411개의 스토리 노드 JSON 파일이 존재하며, 이것만으로 13일치 분량의 게임 콘텐츠가 구성된다.

StoryState가 노드를 진입할 때의 실행 흐름은 다음과 같다.

```

StoryState::LoadNode(id)
→ StoryLoader::Load("Data/story/" + id + ".json")
→ ConditionChecker::Check(node.require, ctx) // 진입 조건 검사
→ EffectInterpreter::Apply(node.effects, ctx) // 진입 이펙트 적용
→ visibleChoices 필터링 (각 선택지 require 평가)
→ UITypewriter로 텍스트 출력

```

`ConditionChecker::Check()`는 조건 배열을 **AND** 평가한다. 하나라도 불충족이면 선택지 자체가 표시되지 않는다. `EffectInterpreter::Apply()`는 17종의 이펙트를 처리한다. 이펙트 타입은 `EffectType` 열거형으로 정의되어 새 타입 추가 시 `Apply()` 내 `switch` 분기만 추가하면 **JSON** 데이터는 즉시 새 이펙트를 사용할 수 있다.

JSON만 편집하면 프로그래머의 개입 없이 새로운 분기를 추가할 수 있다. 조건 타입(`fatigue`, `flag`, `has_item`, `tendency` 등)과 이펙트 타입(`add_flag`, `give_item`, `unlock_achievement` 등)이 명세로 고정되어 있어 실수를 방지하면서도 표현 범위가 넓다.

4. 주요 기능 구현 명세

4.1 콘솔 더블 버퍼링 렌더링

ConsoleDisplay는 width=192, height=54의 CHAR_INFO 배열을 백 버퍼로 사용한다. 각 UIElement는 ConsoleDisplay::Draw() 또는 DrawText()로 백 버퍼에 문자를 기록하고, 프레임 끝에 Present()가 WriteConsoleOutput() API를 한 번만 호출하여 전체 버퍼를 콘솔에 출력한다. 시스템 커서가 보이지 않고 출력이 한 번에 이루어지므로 깜빡임이 없다.

```
void ConsoleDisplay::Present()
{
    COORD size = { (SHORT)width, (SHORT)height };
    COORD origin = { 0, 0 };
    SMALL_RECT region = { 0, 0, (SHORT)(width-1), (SHORT)(height-1) };
    WriteConsoleOutput(hConsole, screen.data(), size, origin, &region);
}
```

4.2 화면 전환 슬라이드 애니메이션

State 전환 시 StateMachine::StartFade()가 호출되면 현재 화면의 스냅샷(TakeSnapshot())을 저장한다. Update()마다 m_fadeProgress가 kFadeSpeed(4.0f/초)만큼 증가하며, RenderWithSlide()는 스냅샷을 아래 방향으로 이동(BlitSnapshot)시키면서 현재 화면과 합성하여 슬라이드 효과를 만든다. 설정에서 screenTransition=false이면 m_fadeProgress를 즉시 1.0으로 설정해 애니메이션을 건너뛴다.

4.3 UTF-8 및 멀티바이트 문자 처리

콘솔에서 한국어·일본어·중국어 등을 정확히 출력하려면 문자 폭(시각적 너비)을 별도로 계산해야 한다. Utils/GetCharWidth와 GetVisualWidth가 이를 담당한다. 한글·한자·일본어 가나 등 전각 문자는 콘솔에서 2칸을 차지하므로 레이아웃 계산 시 바이트 수가 아닌 시각 폭을 기준으로 삼는다. UTF8ToWide는 std::string(UTF-8) → std::wstring(UTF-16) 변환을 수행하며, Windows API의 MultiByteToWideChar를 사용한다.

4.4 저장·불러오기 시스템

게임 상태 전체(PlayerStats, flags, inventory, log, journal, characters, currentNodeId)를 JSON으로 직렬화하여 Data/saves/slot_N.json에 저장한다. 일반 슬롯 3개와 퀵 슬롯 5개를 지원한다.

nlohmann/json의 직렬화 인터페이스를 활용하여 PlayerStats의 각 필드를 JSON 키-값 쌍으로 변환하며, std::unordered_set<string>(flags)과 std::unordered_map<string, int>(inventory)도 JSON 배열·오브젝트로 변환된다. 저장 시 PlayrtStats와 함께 currentNodeId, activeSlot도 함께 직렬화되어 불러오기 시 플레이어가 마지막으로 진행하던 스토리 노드와 슬롯 번호가 그대로 복원된다.

구분	슬롯 수	파일 경로	저장 내용
일반 슬롯	3개	Data/saves/slot_N.json	PlayerStats·flags·inventory·log·journal·characters·currentNodeId
퀵 슬롯	5개	Data/saves/quick_N.json	동일

4.5 다국어 시스템

LocalizationManager는 Data/lang/{언어코드}.json을 로드하여 key→string 맵을 구성한다. 코드 어디서든 L("key") 매크로를 호출하면 현재 설정 언어의 문자열이 반환된다. 설정에서 언어를 변경하면 JSON 파일을 다시 로드하고 UI 전체가 즉시 갱신된다. JSON 파일만 추가하면 새 언어를 등록할 수 있어 실제로 한국어·영어·일본어·중국어·프랑스어 5개 언어가 지원된다.

```
L("ui.new_game") // → "새 게임" (ko) / "New Game" (en) / "新游戏" (zh) ...
```

4.6 업적·저널·인물 시스템

업적 시스템(AchievementManager)은 세이브 슬롯과 독립된 Data/saves/achievements.json에 해제 상태를 저장한다

4.7 PlayerStats 수치 체계

게임의 서사는 플레이어 수치에 의해 결정된다. 구조는 세 계층으로 분리된다.

분류	항목	기본값	설명
개인 수치	fatigue / monitoring / wealth	0 / 0 / 0	피로도·감시 등급·재화
세계 수치	cityOrder / citizenTrust	50 / 50	도시 질서·시민 신뢰
세계 수치	corruption / day / time	0 / 1 / 8	오염도·날짜·시각(0~23)
성향	empathy / coldness / justice	0 / 0 / 0	공감·냉정·정의 (감소 없음)
성향	compliance / suspicion	0 / 0	순응·의심 (감소 없음)

성향(tendency) 수치는 감소하지 않는 일방향 누적값이다. Disco Elysium에서 영감을 받은 이 설계는 플레이어의 선택 이력이 캐릭터 성격으로 굳어진다는 서사를 수치로 표현한다.

5. 팀원별 역할 분담 및 협업 과정

5.1 역할 분담

역할	이름	담당 내용
팀장 / Core 시스템	이형원	Application·GameLoop·StateMachine·EventBus·Context 설계·구현, 콘솔 렌더링, 화면 전환 슬라이드 애니메이션
저장 / 로드	노민지	SaveSlot·LoadSlot·QuickSlot 시스템, 게임 상태 JSON 직렬화·역직렬화
UX / UI / 번역	홍성우	UIButton~UIScreenFader 8종 구현, 한·영·일·중·프 5개 언어 번역
콘텐츠 / 데이터	김세민	스토리 JSON 제작(411개 노드), 아이템·업적 데이터 설계, ConditionChecker·EffectInterpreter 구현

5.2 협업 방식

기능별 브랜치 전략을 사용했다. **feat/·fix/·docs/** 접두사로 브랜치를 구분하고, **Pull Request** 검토 후 **master**에 병합했다. **PR**은 총 4건이 확인된다.

초반에는 각자의 작업 영역이 충돌하거나 의존성 문제가 발생했다. 이를 해결하기 위해 **Context** 기반 의존성 주입 구조를 도입하였고, 이후 각자 **Context** 참조만 받으면 어느 서브시스템에나 접근할 수 있어 협업 효율이 크게 향상되었다.

6. 프로젝트 수행 후기 및 결론

6.1 팀원별 소감

이형원 | 팀장 / Core 시스템

Core 시스템을 설계하면서 **StateMachine**의 스택 구조와 **EventBus**의 **Pub/Sub** 패턴이 얼마나 강력한지를 직접 체감했다. 특히 **PendingStateOps**를 통해 **use-after-free** 문제를 해결하는 과정에서 메모리 관리의 중요성을 다시 깨달았다. 팀장으로서 전체 구조를 설계하고 팀원들이 그 위에서 자유롭게 작업할 수 있도록 기반을 마련하는 것이 어려웠지만, 완성된 결과물을 보며 큰 보람을 느꼈다.

노민지 | 저장 / 로드

저장·불러오기 시스템을 구현하면서 **JSON** 직렬화의 편의성과 함께 슬롯 충돌이나 예외 처리의 중요성을 배웠다. **PlayerStats**, 플래그, 인벤토리, 저널 등 게임 상태 전체를 하나의 파일로 안전하게 직렬화하는 과정이 생각보다 복잡했다. 이 과정에서 **nlohmann/json** 라이브러리의 활용 방법과 데이터 구조 설계의 중요성을 실감했다.

홍성우 | UX / UI / 번역

콘솔이라는 제한적인 환경에서 애니메이션과 **UI**를 구현하는 것이 처음에는 막막했다. 타이프라이터 효과, 슬라이드 애니메이션, 호버 효과를 하나씩 완성해나가면서 큰 성취감을 느꼈다. 5개 언어 번역 작업 중 한글·일본어·중국어의 멀티바이트 문자 폭 처리 문제를 해결하는 과정에서 유니코드와 문자 인코딩에 대해 깊이 이해했다.

김세민 | 콘텐츠 / 데이터

스토리 노드를 **JSON**으로 설계하면서 데이터 주도(**data-driven**) 개발의 장점을 실감했다. 코드 수정 없이 **JSON** 파일만 편집하면 스토리 분기가 바뀌는 구조가 매우 유연하게 느껴졌다.

ConditionChecker와 **EffectInterpreter**를 구현하면서 다양한 조건 타입과 이펙트를 확장 가능하게 설계하는 방법을 배웠고, 콘텐츠 제작자 입장에서 개발 구조를 바라보는 시각도 얻었다.

6.2 배운 객체지향 개념 정리

OOP 개념	적용 사례
추상 클래스 / 다형성	UIElement·State 추상 클래스. UIManager·StateMachine이 구체 타입 없이 가상 함수 호출
합성(Composition)	Context가 EventBus·ConsoleDisplay·SoundSystem 등을 직접 포함. 상속보다 합성 선호 원칙
단일 책임 원칙	StoryLoader(파싱), ConditionChecker(조건 평가), EffectInterpreter(이펙트 적용)으로 역할 분리
개방-폐쇄 원칙	UIElement·State 계층. 새 UI/State 추가 시 기존 코드 수정 불필요
RAII	EventSubscription이 소멸자에서 Unsubscribe() 자동 호출. unique_ptr로 State 소유권 관리
Observer 패턴	EventBus의 Subscribe<T>/Emit<T>. 발행자·구독자 직접 의존 없이 이벤트 통신
State 패턴	StateMachine의 스택 기반 State 관리. 각 State가 입력·갱신·렌더를 독립 처리

7. 참고문헌

외부 라이브러리

[1] nlohmann/json — C++ JSON 파싱 헤더 온리 라이브러리 (MIT License).

<https://github.com/nlohmann/json>

[2] miniaudio — 단일 헤더 오디오 재생·캡처 라이브러리 (MIT / Public Domain). <https://miniaud.io>

벤치마킹 게임

[3] Papers Please (2013) — Lucas Pope, 3909 LLC — 관료 시스템 세계관 및 스탯 설계 참고.

<https://www.papersplea.se>

[4] Disco Elysium (2019) — ZA/UM — 성향 누적 시스템 및 선택지 잠금 설계 참고.

<https://zaumstudio.com>

[5] Orwell (2016) — Osmotic Studios, Fellow Traveller — 파일 처리 UX 및 감시 시스템 참고.

https://store.steampowered.com/app/491950/Orwell_Keeping_an_Eye_On_You/

[6] Roadwarden (2022) — Moral Anxiety Studio — 텍스트 RPG UI/UX 구성 참고.

<https://moralanxiestystudio.com>

C++ 표준 및 EventBus 관련

[7] std::type_index — cppreference. https://en.cppreference.com/w/cpp/types/type_index

[8] typeid 연산자 — cppreference. <https://en.cppreference.com/w/cpp/language/typeid>

[9] std::function — cppreference. <https://en.cppreference.com/w/cpp/utility/functional/function>

[10] 이벤트 버스 패턴 — levell1.github.io.

https://levell1.github.io/design%20pattern/MUnity-EventBus_Pattern/

개발 참고 자료

[11] 함수 템플릿 — blockdmask.tistory.com. <https://blockdmask.tistory.com/43>

[12] JSON 기반 언어 파일 구조 참고 — 한마포.

https://www.koreaminecraft.net/mod_lecture/3159766

[13] ASCII Art 렌더링 참고 — codingapple.com. <https://codingapple.com/blog/ascii-rendering/>